

Descripción de la Persistencia de Datos

Proyecto MediFlow — DSY1106 Desarrollo Fullstack III — Evaluación Parcial N°3

1. Estrategia general

El sistema MediFlow reemplaza una base de datos monolítica de 2 TB por un esquema de persistencia descentralizado, donde **cada microservicio administra su propia base de datos PostgreSQL independiente**. Este principio (database-per-service) elimina el punto único de falla del monolito original y permite escalar, respaldar y desplegar cada servicio por separado. No existe una base de datos central compartida.

Microservicio	Base de datos	Tecnología de acceso
Core HCE	mediflow_hce (PostgreSQL)	JPA / Spring Data (ORM)
Laboratorios	mediflow_lab (PostgreSQL)	Procedimientos Almacenados (SPs)

2. Microservicio Core HCE — Persistencia vía JPA

El microservicio de Historias Clínicas Electrónicas utiliza **JPA (Java Persistence API)** a través de Spring Data. Las entidades Paciente e HistoriaClinica se mapean a tablas mediante anotaciones (@Entity, @Table, @ManyToOne). Spring Data genera automáticamente las operaciones CRUD y las consultas derivadas a partir de los nombres de método del repositorio.

Se eligió JPA para este servicio porque las Historias Clínicas requieren operaciones CRUD frecuentes y relaciones entre entidades (un paciente posee múltiples registros), un escenario donde el mapeo objeto-relacional reduce código repetitivo y mejora la mantenibilidad.

Ejemplo de consulta derivada (Spring Data genera la implementación):

```
List<HistoriaClinica> findByPacienteIdOrderByFechaRegistroDesc(Long pacienteId);
```

3. Microservicio Laboratorios — Procedimientos Almacenados

El módulo de Laboratorios persiste sus datos mediante **Procedimientos Almacenados (SPs)** ejecutados directamente en PostgreSQL e invocados desde el repositorio con @Query nativa y la sentencia CALL. Esta decisión demuestra el uso de una tecnología de persistencia distinta a la del otro microservicio (requisito de la rúbrica) y resulta apropiada cuando la lógica de inserción y cambio de estado conviene mantenerse del lado del motor de base de datos.

Procedimientos definidos en schema.sql:

- sp_registrar_solicitud: inserta una nueva solicitud de estudio.
- sp_actualizar_estado: cambia el estado de una solicitud.
- fn_solicitudes_urgentes: función que retorna las solicitudes urgentes pendientes.

Invocación desde el repositorio (consulta nativa):

```
@Query(value = "CALL sp_registrar_solicitud(:pacienteId, :tipoEstudio, :prioridad, :medico)", nativeQuery = true)
```

4. Justificación de la elección (software open source)

PostgreSQL es un motor de base de datos relacional open source (licencia PostgreSQL License), maduro, sin costos de licenciamiento y con amplio soporte de la comunidad. Es la base de datos que el cliente ya utiliza, por lo que su adopción no introduce nuevas dependencias propietarias. Tanto Spring Data JPA como el driver JDBC de PostgreSQL son igualmente open source, garantizando una solución completa sin costos de licencia y alineada con los requisitos del curso.